

```
#!/usr/bin/env python3
```

```
"""
```

Concierge Meta-Prompt Compiler (concierge_prompt_compiler.py)

Derived from: W5/H-(6-files) & CC_Planning-session-UNORGANIZED.txt

Purpose:

Transforms raw, unorganized voice/speech transcripts or stream-of-consciousness operator inputs into crisp, structured, executive Markdown meta-prompts suitable for frontier models (Claude 3.7 / Gemini 2.5) without typos or run-on syntax.

Usage:

```
python3 concierge_prompt_compiler.py --input "raw_transcript.txt" --output "meta_prompt.md"
```

```
"""
```

```
import sys
```

```
import argparse
```

```
import re
```

```
from pathlib import Path
```

```
def sanitize_raw_transcript(raw_text: str) -> str:
```

```
    # 1. Normalize line endings and whitespace
```

```
    text = re.sub(r'\r\n|\r', '\n', raw_text)
```

```
    text = re.sub(r'[ \t]+' , ' ', text)
```

```
    # 2. Strip conversational filler phrases
```

```
    fillers = [
```

```
        r'\b(um|uh|like|you know|basically|i mean|so yeah|right|kind of|sort of)\b',
```

```
        r'\b(i was thinking|i reckon|or whatever you call that|blah blah blah)\b'
```

```
    ]
```

```
    for pattern in fillers:
```

```
        text = re.sub(pattern, '', text, flags=re.IGNORECASE)
```

```
    # 3. Clean up punctuation and orphaned commas
```

```
    text = re.sub(r'\s*,\s*', ', ', text)
```

```
    text = re.sub(r'\s*\.\s*', '. ', text)
```

```
    text = re.sub(r'\s+', ' ', text).strip()
```

```
    return text
```

```
def compile_meta_prompt(clean_text: str, context: str = "") -> str:
```

```
    lines = [
```

```
        "# COMPILED EXECUTIVE META-PROMPT",
```

```
        "***Source:** Concierge Speech-to-Text Ingress Layer",
```

```
        "***Target:** Frontier Reasoning / Code Execution Engine",
```

```

    "___",
    "## Primary Objective",
    clean_text,
    """
    ## Execution Constraints & Invariants",
    "- Strictly Non-Destructive: Do not modify or delete original source files.",
    "- Clean Markdown: Structure output with clear section headers and concise markdown
tables.",
    "- Deterministic Output: Provide concrete file paths, valid code syntax, and verifiable
outputs.",
    "- Grounded Links: Use exact drive URLs when referencing system artifacts.",
    """
]
if context:
    lines.extend([
        "## Injected Context",
        context,
        ""
    ])
lines.append("## Requested Deliverables")
lines.append("1. Produce the structured artifact/code.")
lines.append("2. Output verification checklist and file location summary.")
return "\n".join(lines)

```

```

def main():
    parser = argparse.ArgumentParser(description="Compile raw speech transcript into executive
meta-prompt.")
    parser.add_argument("--input", "-i", type=str, help="Path to input text file containing raw
transcript.")
    parser.add_argument("--text", "-t", type=str, help="Direct raw transcript string.")
    parser.add_argument("--context", "-c", type=str, default="", help="Optional system context to
append.")
    parser.add_argument("--output", "-o", type=str, help="Path to write compiled meta-prompt
markdown.")
    args = parser.parse_args()

    raw = ""
    if args.input:
        raw = Path(args.input).read_text(encoding="utf-8")
    elif args.text:
        raw = args.text
    else:
        raw = sys.stdin.read()

```

```
if not raw.strip():
    print("Error: No transcript input provided.", file=sys.stderr)
    sys.exit(1)

cleaned = sanitize_raw_transcript(raw)
meta_prompt = compile_meta_prompt(cleaned, args.context)

if args.output:
    Path(args.output).write_text(meta_prompt, encoding="utf-8")
    print(f"[✓] Meta-prompt compiled successfully to: {args.output}")
else:
    print(meta_prompt)

if __name__ == "__main__":
    main()
```